# Lecture 3

# Memory Hierarchy

Mingyu Gao

gaomy@tsinghua.edu.cn

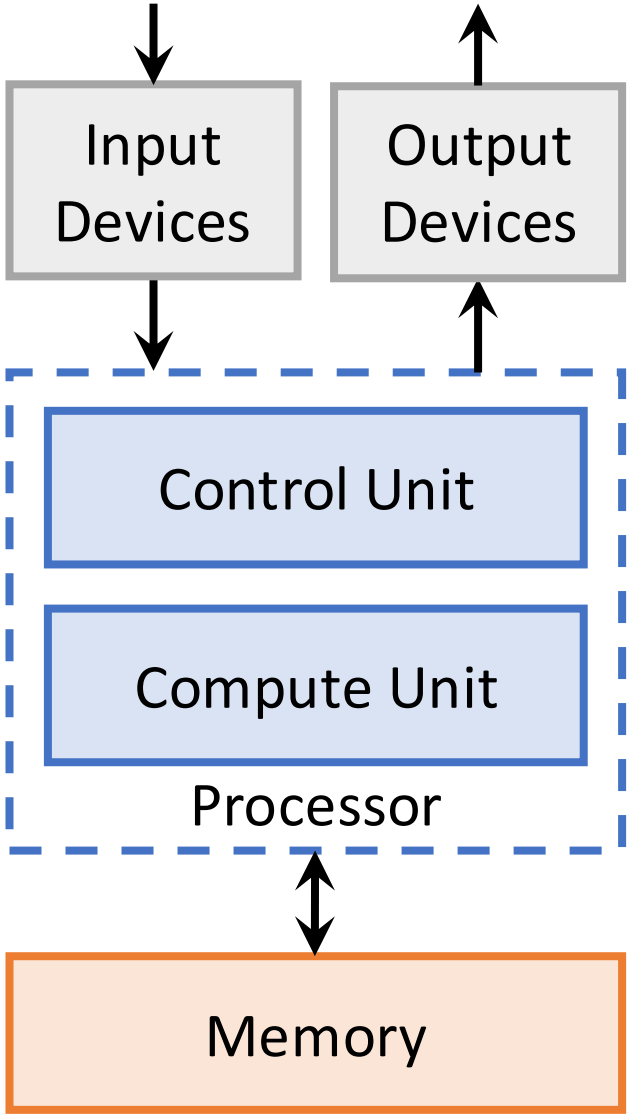
*IIIS, Tsinghua*

# Announcement

- Lab 1 will be released this week
  - About performance optimization
  - You have about 3 weeks; refer to the deadline on 网络学堂
  - Start early as HW 1 will also be out next week!

- TA will introduce Lab 1 in the review session

# Recall: The Big Picture of Computer Systems




- Von Neumann architecture
- Separate data processing (processors) and data storage (memory)
- We have discussed processor hardware design and instructions executing on processors
- Now we focus on data storage

# Primary vs. Secondary Storage

- Primary storage, a.k.a., main memory, or simply memory
  - Fast but volatile (lose contents after powered off)
  - Used to temporarily store data during program execution
  - Accessed through load and store instructions issued by processor (hardware)
  - Implemented with DRAM and optimized by SRAM caches
- Secondary storage, a.k.a., external storage (treated as I/O devices)
  - Slow but non-volatile
  - Used to permanently keep data required/generated (input/output) by programs
  - Accessed through and managed by file systems in OS (software)
  - Implemented with disks and/or SSDs

- We talk about memory today; secondary storage will be discussed in later lectures about OS

# Why Separate Primary and Secondary Storage

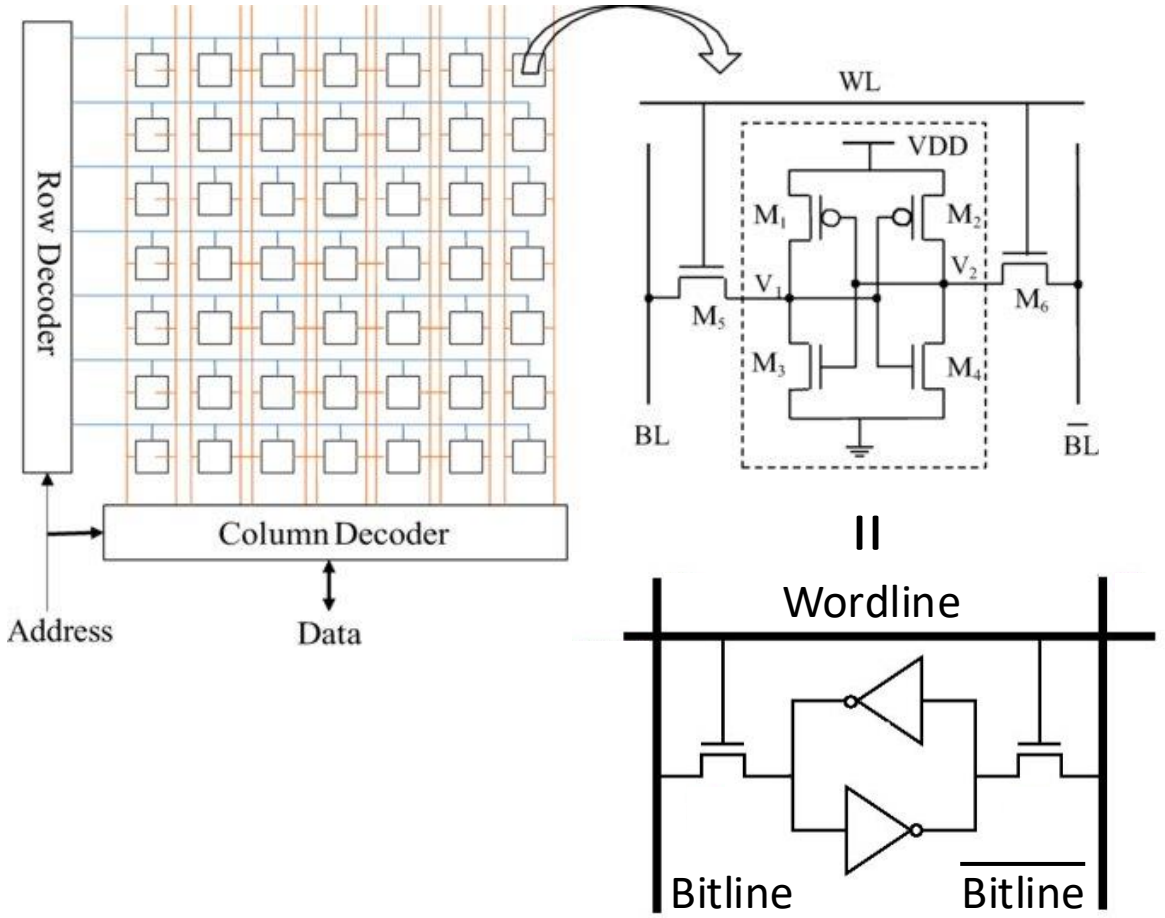
- During program execution, we want fast data accesses but do not need persistency, so we use faster but volatile memories, managed by hardware
  - Mostly for intermediate and temporary data
- Permanently stored data usually need stronger data persistency and consistency, which is hard to guarantee by hardware and needs software techniques
  - See the later lecture of database and transactions
- Each program instance has its private memory space, but all programs share data in secondary storage, e.g., one program can use another program's output as input
  - Need a universal format in the secondary storage, while each program can choose its own in-memory data structures for best efficiency
- And more other reasons ……

# Today's Outline

- Memory technology choices and performance-capacity tradeoff
- Program locality and the idea of memory hierarchy
- Processor caches
  - Cache performance analysis
  - Cache organization: direct-mapped, fully associative, set associative
  - Replacement policy
  - Write policy
  - ……
- Multi-level cache hierarchy and multi-core caches
- Cache-friendly code optimization

# SRAM (Static Random Access Memory)

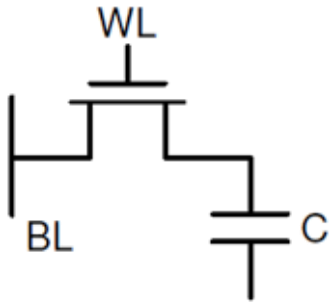
- RAM = able to access any address
- 2D array
  - Row and column address decoders
- ($n$+$m$)-bit address → row & col addr
  - $n$-bit row addr activates 1 of $2^n$ wordlines
  - $m$-bit col addr selects $w$ of $2^m \times w$ bitlines



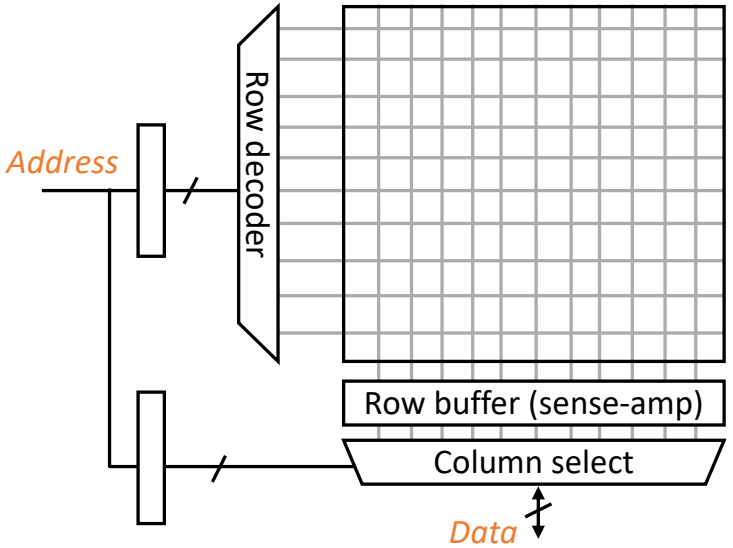

- Characteristics
  - ✓ Use same technology as processors; could be on the same chip
  - ✓ Similarly fast access speed as processors
  - ✗ Each cell is large (6T) → small capacity

# DRAM (Dynamic RAM)

- Similar 2D array and row/column address
- Capacitor charged/discharge → store bit 1/0
  - Static vs. dynamic circuits: data represented by voltage vs. by charge

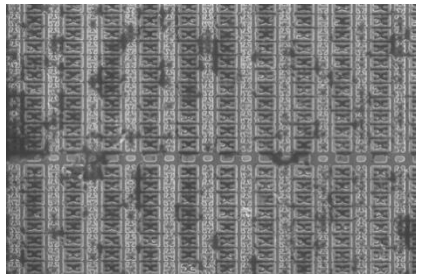



- Characteristics
  - ✓ Each cell is small (1T1C) → large capacity
  - ✗ Different from processor technology; separate chips
  - ✗ Slower access due to charge and discharge
  - ✗ Cell access is destructive; need extra complex circuits



# Memory Technology Tradeoff

- Tradeoff: large memory capacity vs. fast memory access speed
  - No memory technology provides both large capacity and fast speed!
- Fundamental physical principles decide time and energy
  - Larger arrays need longer time and higher energy to just get to the data location
  - Denser cells usually forbid strong driving logic needed for fast access, and/or rely on more complex phenomena that are slower and more expensive to toggle bits



SRAM
~ 1 ns, ~ 1 MB



DRAM
~ 50 ns, ~ 10 GB



SSD
~ 10 µs, ~ 1 TB



Hard Disk
~ 1 ms, ~ 10 TB



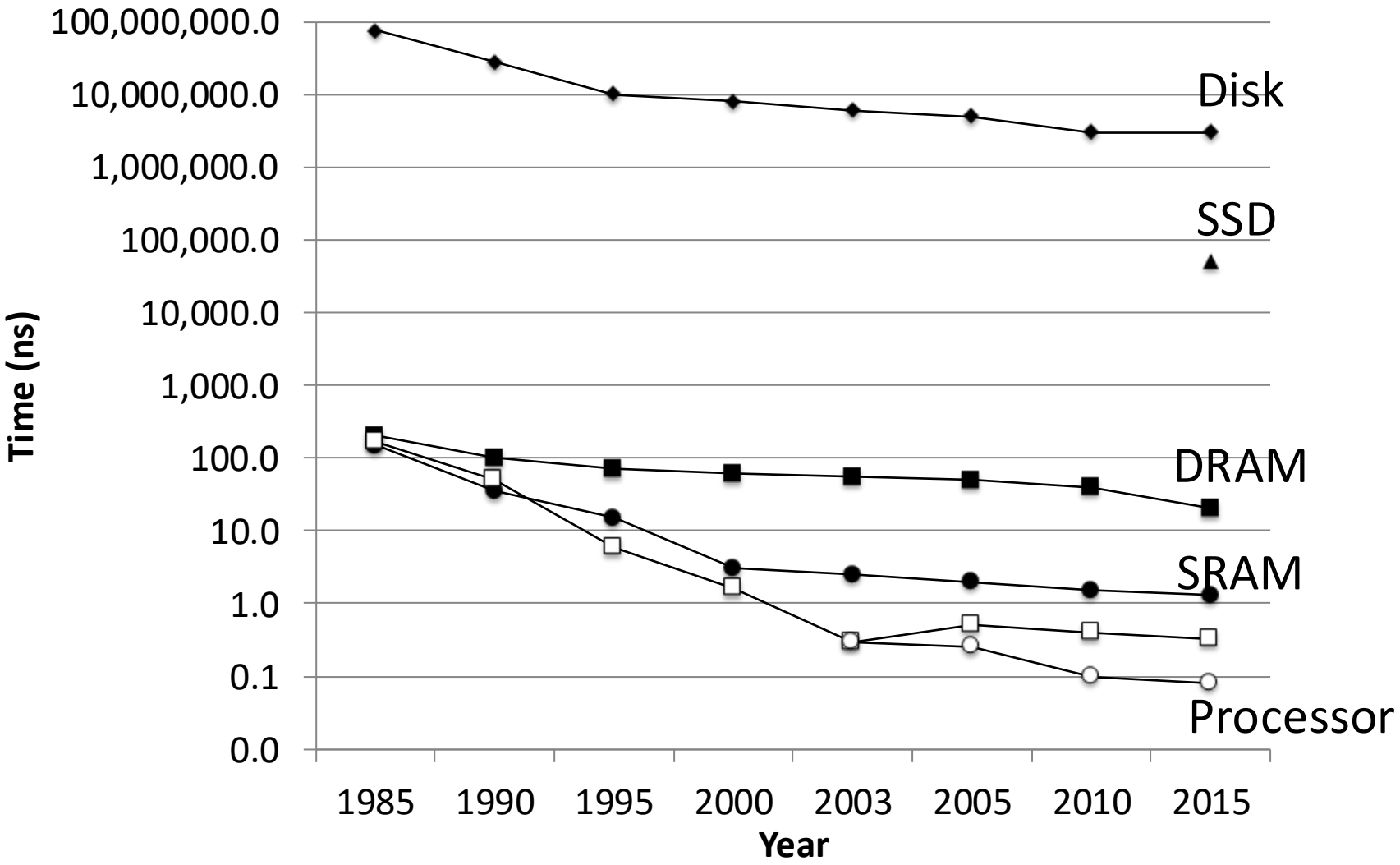
NAS
~ 10 ms, ~ 100 TB

*Faster*
*Smaller*



*Larger*
*Slower*

# And What's Worse: Processor-Memory Gap




Memory technologies have been improving more slowly than processors, resulting in an increasingly larger performance gap between processor and memory

# How Could We Win Both?

- A great example of how system innovations overcome technology limitations

- Systems should be optimized for common cases, not worst cases
  - A *large* memory system only needs to be *fast* for typical programs
  - Q: What did we do to alleviate the issue of slow memory in the last lecture?

- Our approach
  - Discover common patterns in typical programs

  → Programs have temporal and spatial locality
  - Make use of these patterns to apply optimizations

  → Caches and memory hierarchies

# Today's Outline

- Memory technology choices and performance-capacity tradeoff
- Program locality and the idea of memory hierarchy
- Processor caches
  - Cache performance analysis
  - Cache organization: direct-mapped, fully associative, set associative
  - Replacement policy
  - Write policy
  - ……
- Multi-level cache hierarchy and multi-core caches
- Cache-friendly code optimization

# Locality

- Principle of locality
  - Programs work on a relatively small portion of data at any time
  - Can predict data accessed in near future by looking at recent past accesses

- Temporal locality (locality in time)
  - If an item has been referenced, it will tend to be referenced again soon
- Spatial locality (locality in space)
  - If an item has been referenced, nearby items will tend to be referenced soon



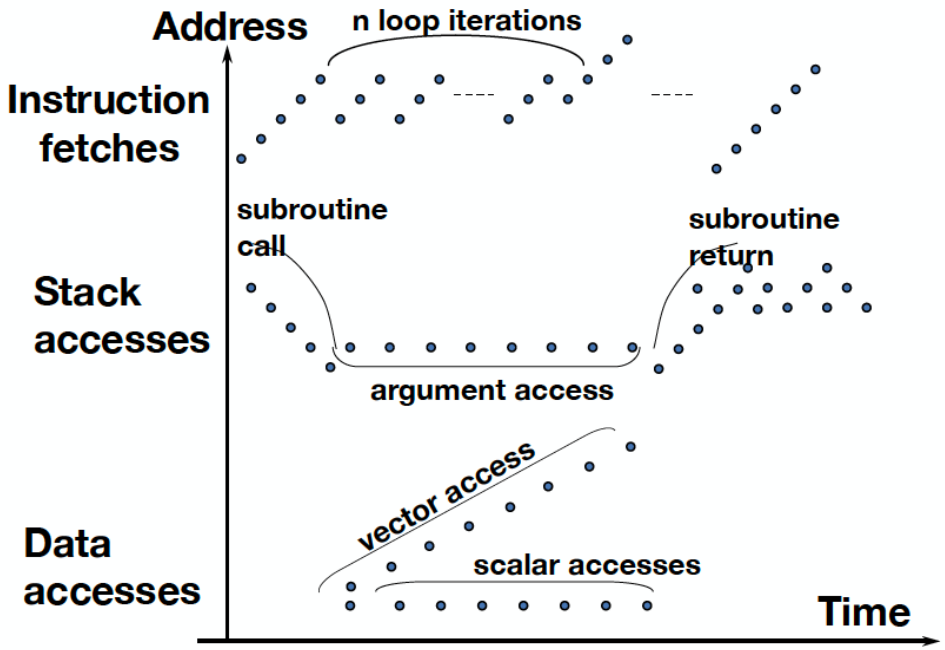
```
l2 = p.x * p.x + p.y * p.y;
d.x = p.x - q.x;
d.y = p.y - q.y;
```



```
for (i = 0; i < N; i++)
    a[i] = ...
```

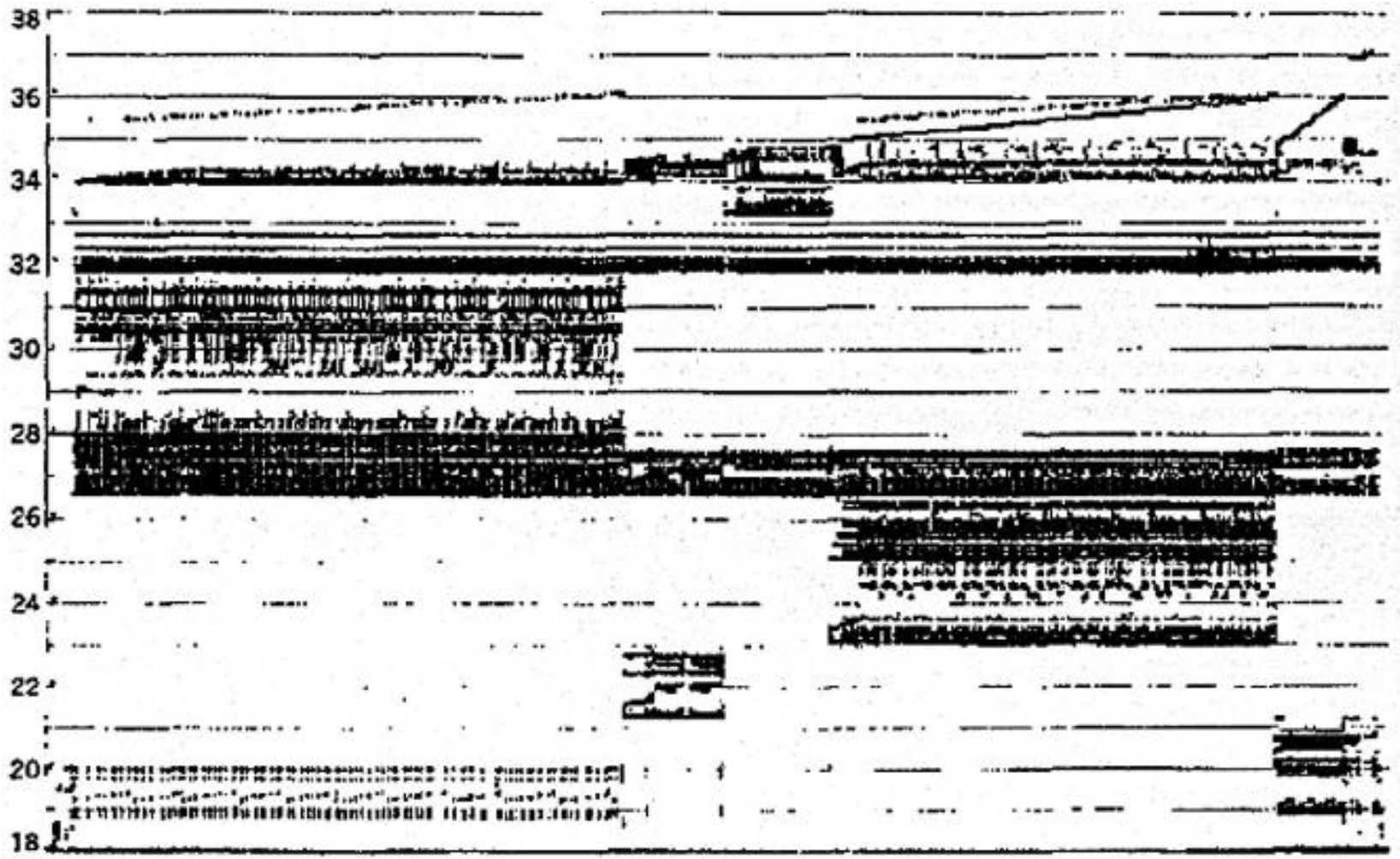
# Locality Example: Data and Instructions

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```




- Data references
  - Spatial locality: reference array elements
  - Temporal locality: reference scalar variable in each iteration
- Instruction references
  - Spatial locality: fetch instructions in sequence
  - Temporal locality: loops

# Real Memory Reference Patterns




Q: Where can we see temporal and spatial locality?

Hatfield, Gerald. *Program Restructuring for Virtual Memory*. IBM Systems Journal 10(3): 168-192 (1971)

# Programs Do Not Always Have Locality

- Q: When do we *not* have locality?
- A common example: pointer chasing
  - Linked lists, trees, graphs, …
  - Every time you follow a pointer, it is to an unrelated location; no spatial locality
  - If you do not chase the pointers again, no temporal locality either

- When algorithm efficiency conflicts with system efficiency
  - Pointers allows many O(1) or O(log n) operations, e.g., lookup, insert
  - But pointers result in higher constant factors in front of the asymptotic terms
- Both simple and advanced data structures could work best in reality, depending on problem sizes and system configurations

# Writing Code with Good Locality

- Being able to look at code and get a qualitative sense of its locality is a key skill for a professional programmer

- Q: How to improve locality for the following code pieces?

```
int sum(int a[M][N]) {
  int s = 0;
  for (int j = 0; j < N; j++)
    for (int i = 0; i < M; i++)
      s += a[i][j];
  return s;
}
```

```
void matmul(int a[M][D], int b[D][N],
            int c[M][N]) {
  for (int i = 0; i < M; i++)
    for (int j = 0; j < N; j++)
      for (int k = 0; k < D; k++)
        c[i][j] += a[i][k] * b[k][j];
}
```

- We will discuss more about this later

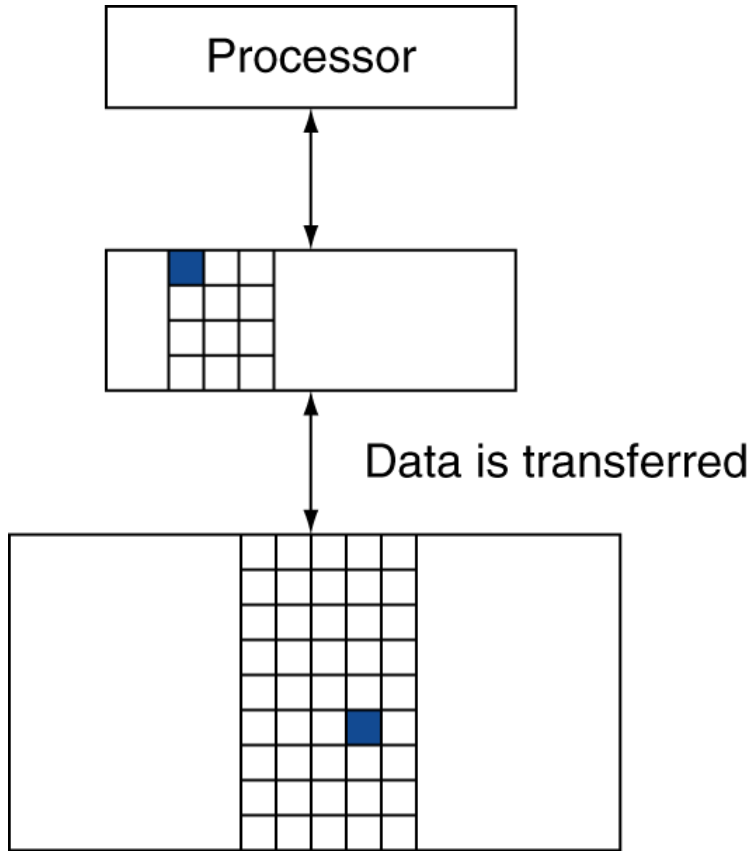
# The Opportunity

- Some fundamental and enduring properties of hardware and software
  - Fast memory technologies cost more per byte, have less capacity, and require more power
  - Well-written programs tend to exhibit good locality

- These fundamental properties complement each other beautifully
- They suggest an approach for organizing memory and storage systems into a memory hierarchy

# Exploiting Locality: Memory Hierarchy

- Store everything on large but slow memory
  - E.g., disks or SSDs
- Copy recently accessed & nearby data to smaller but faster memory
  - E.g., main memory implemented with DRAM
  - Still off-chip w.r.t. processor
- Copy more recently accessed & nearby data to smaller, faster, on-chip caches
  - Implemented with SRAM
  - On the same chip with processor
- Caches can be further split to multiple levels



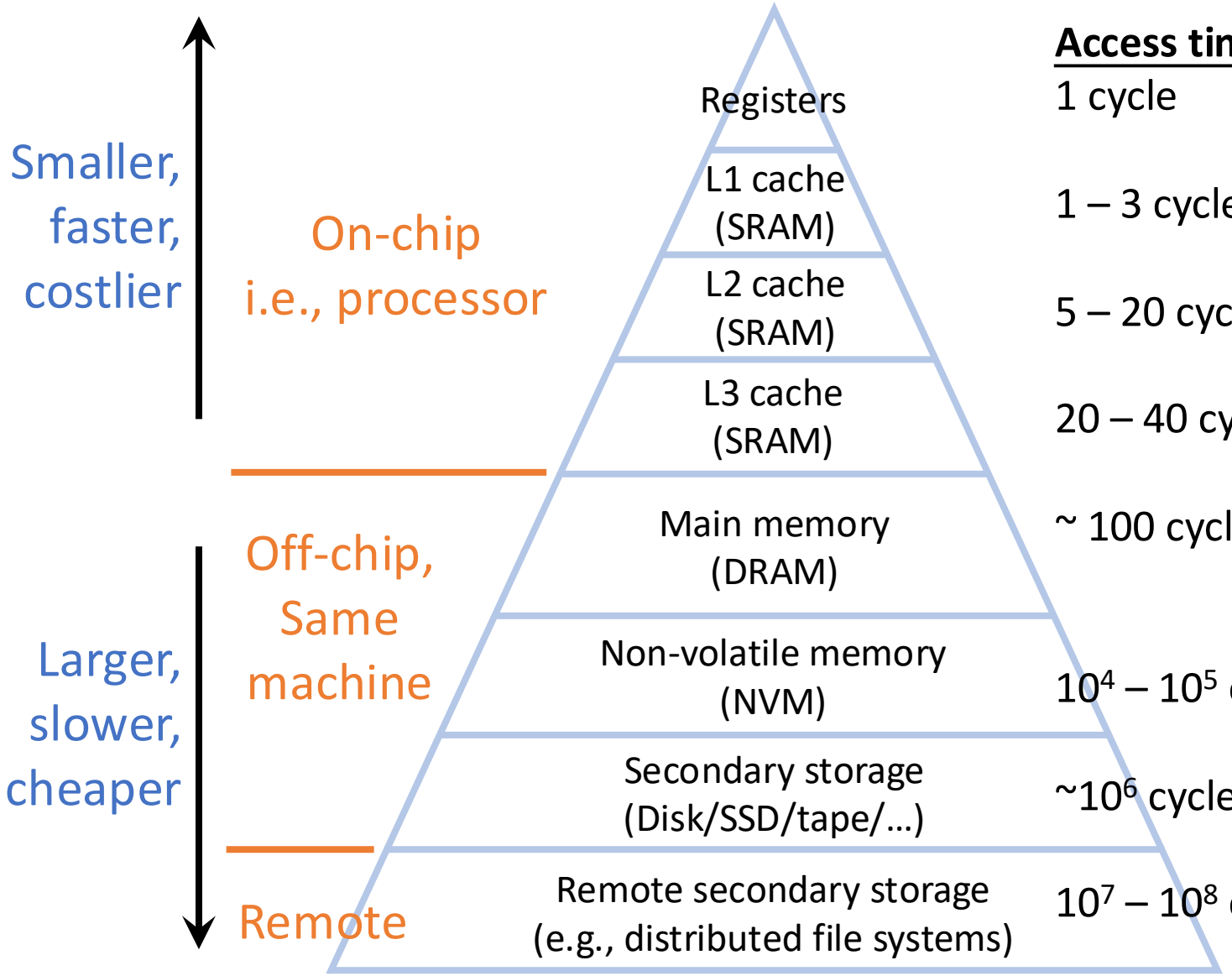
# Memory Hierarchy

- Fundamental idea: the faster, smaller device at level *k* store a subset of data for the larger, slower device at level *k* + 1

- Why does it work?
- Due to locality, programs tend to access a small subset (e.g., 20%) of data (in level *k*) more often (e.g., 80%) than the entire set (in level *k* + 1)
  - Level *k* can be smaller and faster
  - Level *k* + 1 can be slower, and thus larger and cheaper per bit

- Memory hierarchy creates a virtual instance of storage that …
  - Costs as much as the cheap storage near the bottom, but …
  - Serves data to programs at the speed of the fast storage near the top

# Memory Hierarchy Pyramid



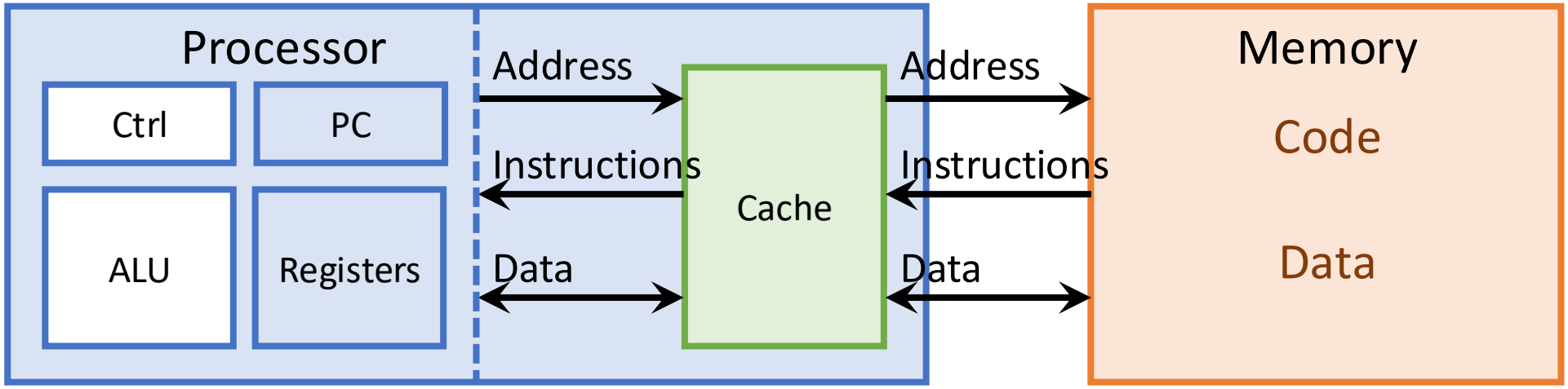

| Access time | Capacity | Managed by |
|---|---|---|
| 1 cycle | ~500 B | software/compiler |
| 1 – 3 cycles | ~64 kB | hardware |
| 5 – 20 cycles | 256 kB – 2 MB | hardware |
| 20 – 40 cycles | 20 – 50 MB | hardware |
| ~ 100 cycles | 10 – 100 GB | software/OS |
| $10^4$ – $10^5$ cycles | 100 GB – 1 TB | software/OS |
| ~$10^6$ cycles | 1 – 10 TB | software/OS |
| $10^7$ – $10^8$ cycles | > 100 TB | software/network |

# Today's Outline

- Memory technology choices and performance-capacity tradeoff
- Program locality and the idea of memory hierarchy
- Processor caches
  - Cache performance analysis
  - Cache organization: direct-mapped, fully associative, set associative
  - Replacement policy
  - Write policy
  - ……
- Multi-level cache hierarchy and multi-core caches
- Cache-friendly code optimization

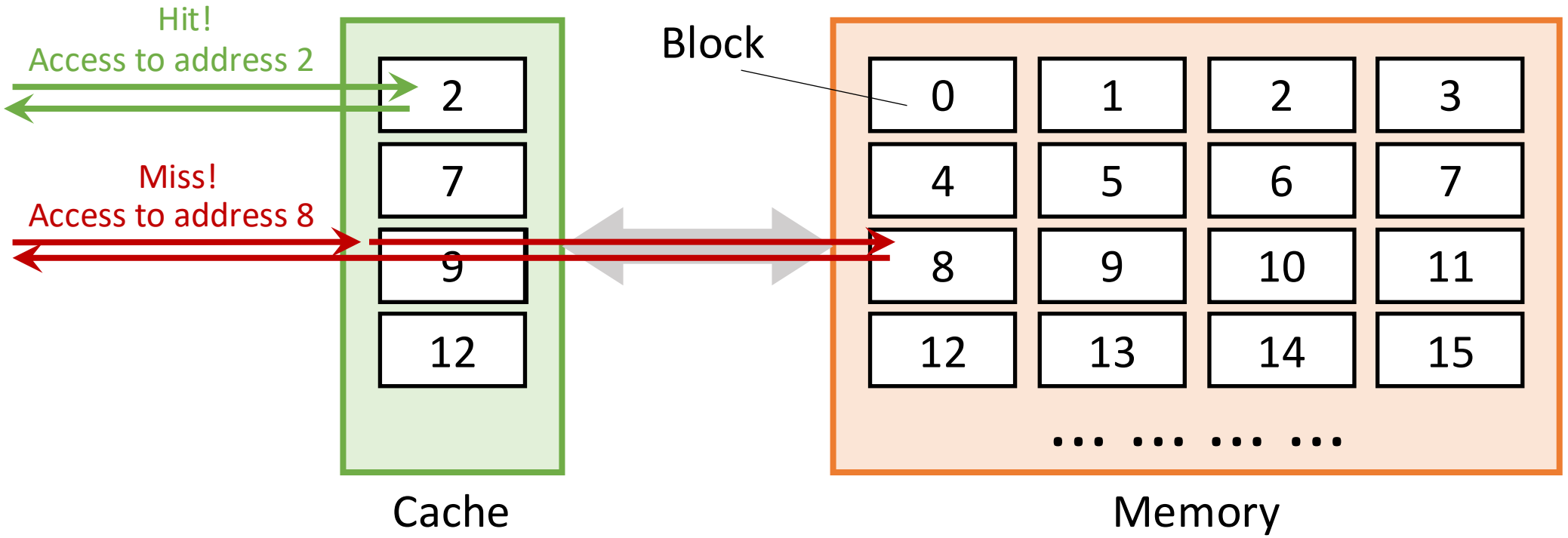
# Caches

- Caches are implemented as processor components
  - Relatively small capacity, but very fast access speed for data in the cache
  - Managed by hardware, and invisible to software: processor issues same load/store instructions as before
- Almost all modern processors use caches
- To make use of caches, write cache-friendly code with good locality



# Concepts in Caches

- Data is transferred in unit of blocks, a.k.a., cachelines (tens of bytes)
- Cache hit: data found in the cache; serve with short latency
- Cache miss: data not found in the cache; need to fetch the block from memory, and may replace a block in the cache



# Cache Performance

- Hit rate: percentage of accesses that hit
- Miss rate = 1 – hit rate
- Hit latency = data access latency on a hit = time to access cache
  - Faster than accessing memory, since cache is smaller
- Miss penalty = overhead of fetching data from memory on a miss
  - = time to access memory + time to deliver to cache (+ time to replace in cache)
    - Q: Do we need to include the last term?
  - This is in addition to hit latency
    - Q: Why?
  - Cache is designed so that hit latency << miss penalty
- AMAT (average memory access time) = hit latency + miss rate × miss penalty

# Cache Performance Examples

- Typical numbers
  - Hit rate of first-level caches: 80% – 95%
  - Hit latency of first-level caches: 1 – 2 ns
  - Miss penalty to memory: 100 – 200 ns
- Huge difference between a hit and a miss, up to 100x

- Example: hit latency = 1 ns, miss penalty = 100 ns
  - With 97% hit rate: AMAT = 1 + 0.03 * 100 = 4 ns
  - With 99% hit rate: AMAT = 1 + 0.01 * 100 = 2 ns
  - 99% hit rate is 2x, not 2%, better than 97%!
- This is why miss rate is preferred to hit rate

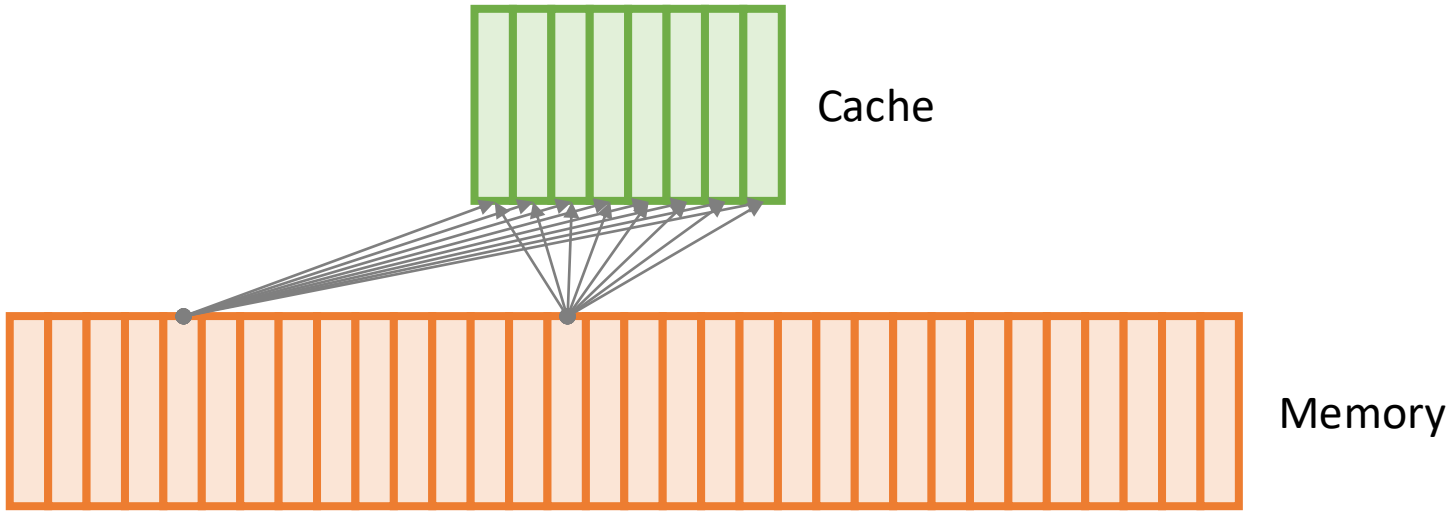
# Key Design Considerations for Caches

- Where to look up and insert a block? --- Cache organization
- Which block to replace to make room? --- Replacement policy
- How to handle writes? --- Write policy

# Fully Associative Caches

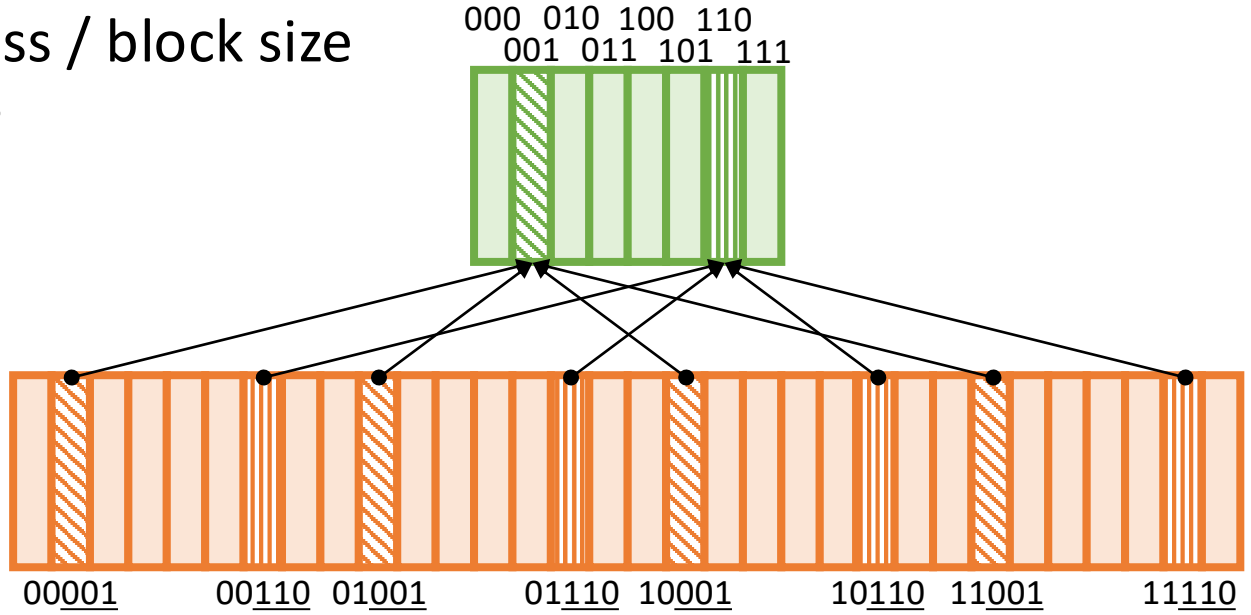
- For a load/store to a specific address, how to find data in the cache?

- Choice 1 – fully associative: a data block can be put anywhere in cache, i.e., use any available location to store a block
  - Pros: have the maximum utilization to cache data, i.e., low miss rate
  - Cons: slow to look up a block (must look at every location), i.e., high hit latency



# Direct-Mapped Caches

- Choice 2 – direct-mapped: each block has a designated location, which is fully determined by its memory address
  - Pros: fast to lookup, i.e., low hit latency
  - Cons: more conflicts between blocks, i.e., high miss rate
- Index of a block (position in cache) = block address % # blocks in cache = lower bits in block address (when # blocks is a power-of-2)
  - Block address = (byte) address / block size
  - Q: Why lower bits for index?




*Note*:
- Address: position in memory
- Index: position in cache

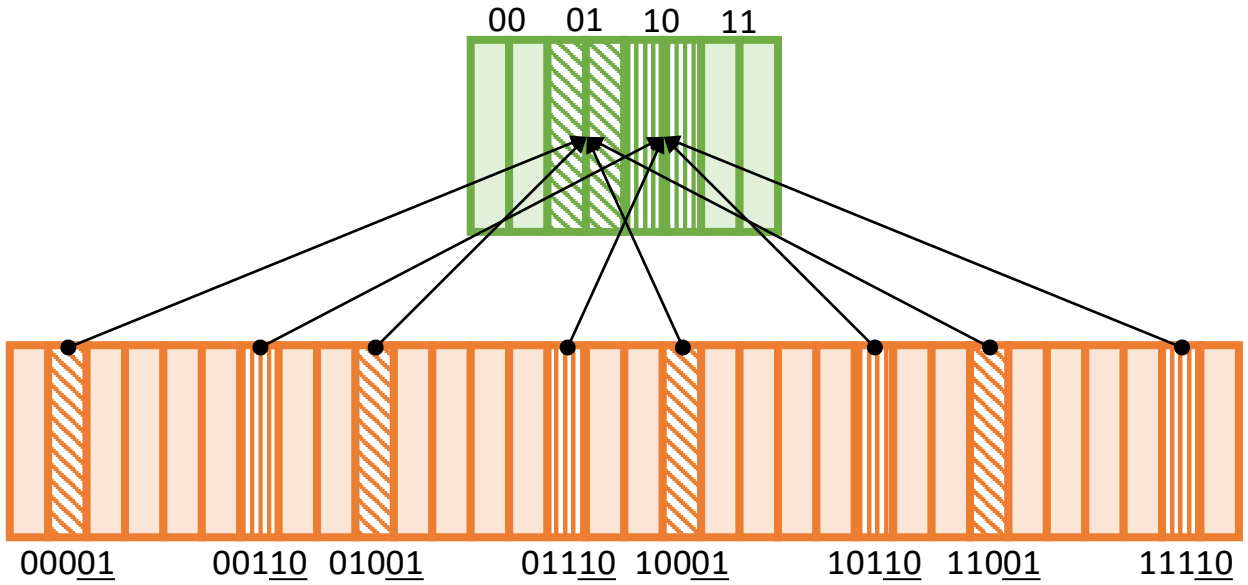
# Address Bits

- Example
  - 8-bit (byte) address
  - 2 bytes/block
  - 4 blocks in cache
  - Direct-mapped

| Address | Block address | Block offset | Index |
|---|---|---|---|
| 00000 00 0 | 0000000 | 0 | 00 |
| 00000 00 1 | | 1 | |
| 00000 01 0 | 0000001 | 0 | 01 |
| 00000 01 1 | | 1 | |
| 00000 10 0 | 0000010 | 0 | 10 |
| 00000 10 1 | | 1 | |
| 00000 11 0 | 0000011 | 0 | 11 |
| 00000 11 1 | | 1 | |
| 00001 00 0 | 0000100 | 0 | 00 |
| 00001 00 1 | | 1 | |
| 00001 01 0 | 0000101 | 0 | 01 |
| 00001 01 1 | | 1 | |
| … | | … | … |

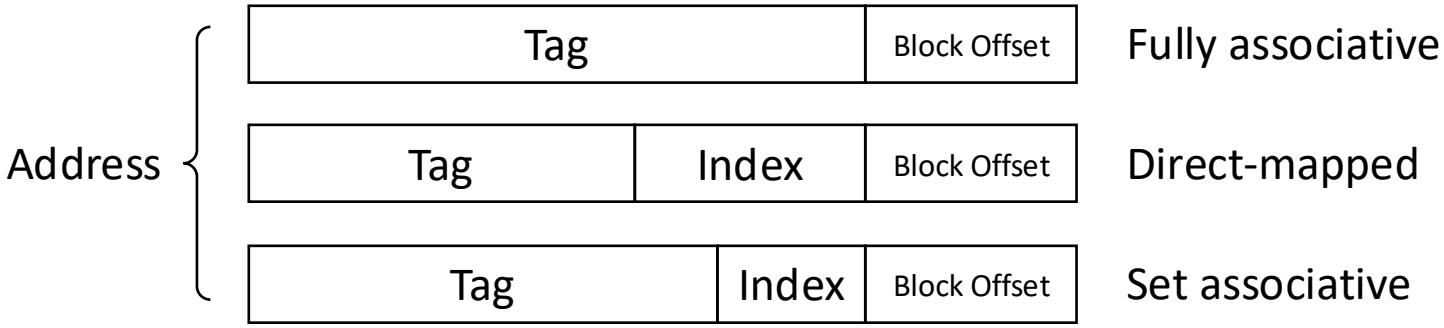
# Set-Associative Caches

- Choice 3 – set-associative: a block can go to one of *N* locations (a set)
  - *N* is called the associativity of the cache; *N*-way set-associative cache
  - Set index = block address % # sets = lower bits in block address
  - All locations in a set must be searched
- Middle ground between direct-mapped and fully associative
  - Direct mapped: *N* = 1 (1-way set-assoc); fully associative: *N* = # blocks in cache



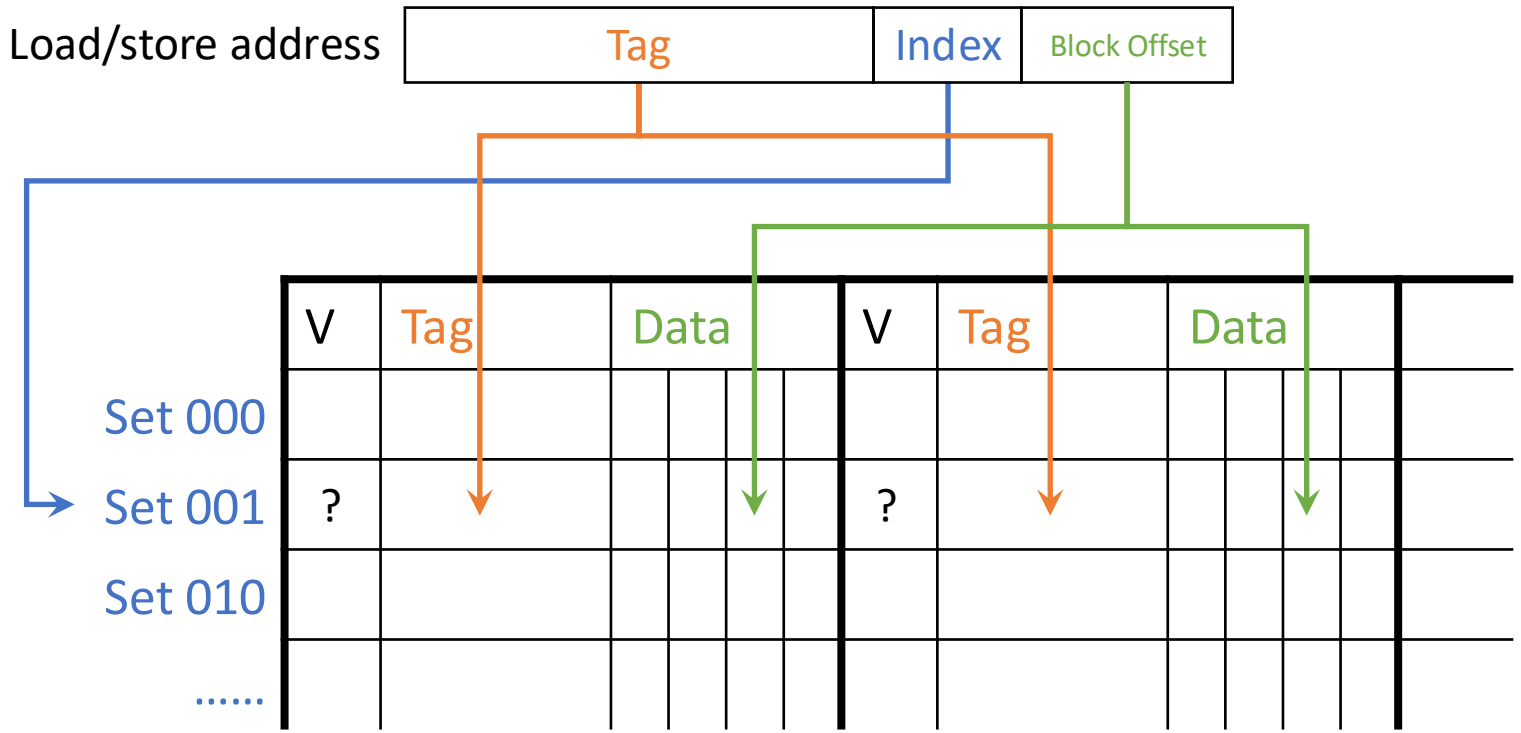
# Tags and Valid Bits

- Each cache location could hold many blocks with different addresses; how do we know whether it has data, and which block it is?
- Valid bit: 1 = has data, 0 = empty
- Tag: also store (part of) block address with data
  - Do not need to store index bits or block offset bits; Q: Why?



# Physical Structure of Caches

- A cache entry = data block + tag + valid bit + (more metadata)
- Access steps
  - Decompose address bits into tag, index, offset
  - Use index to locate a set
  - For each entry in the set
    - Check if valid bit is 1
    - Compare if tag matches
  - If found one such entry, hit, return data
    - Use offset to select bytes
  - If none is found, miss, fetch data from memory, return data, and replace



# Associative Cache Example

All have 8 blocks, but different num. of ways

**One-way set associative (direct mapped)**

| Block | Tag | Data |
|---|---|---|
| 0 | | |
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |
| 6 | | |
| 7 | | |

**Two-way set associative**

| Set | Tag | Data | Tag | Data |
|---|---|---|---|---|
| 0 | | | | |
| 1 | | | | |
| 2 | | | | |
| 3 | | | | |

**Four-way set associative**

| Set | Tag | Data | Tag | Data | Tag | Data | Tag | Data |
|---|---|---|---|---|---|---|---|---|
| 0 | | | | | | | | |
| 1 | | | | | | | | |

**Eight-way set associative (fully associative)**

| Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data | Tag | Data |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | | | | | | | | | | | |

# Cache Organization Example

- A cache has 1024 entries, and uses a block size of 64 bytes. Each memory address is 48 bits.
- What is the data capacity of the cache?
- How many bits do tag, index, and block offset fields have, if the cache is
  - Fully-associative
  - Direct-mapped
  - 4-way set-associative

# The 3Cs of Cache Misses

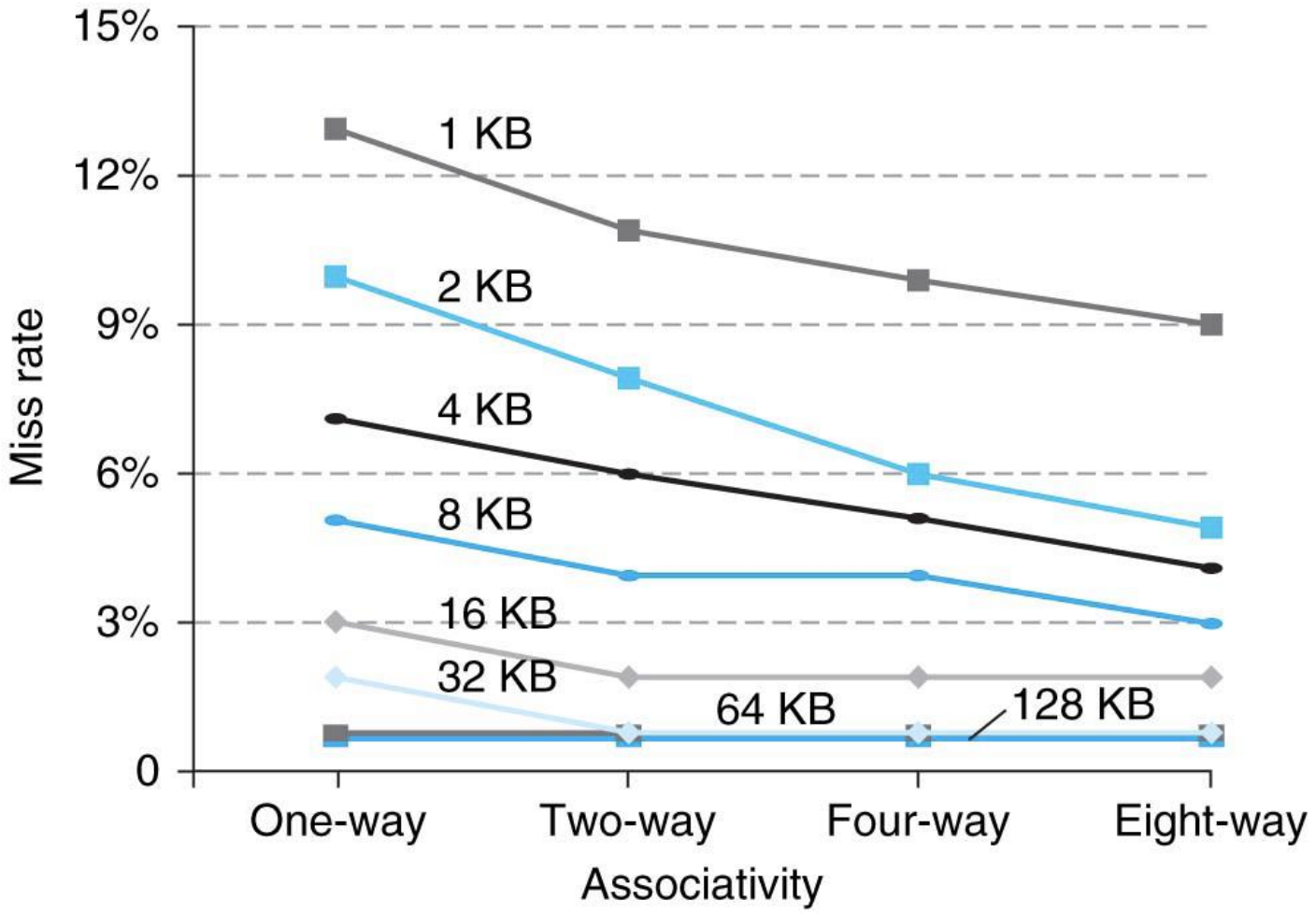
- Why does an access miss in the cache?

- Compulsory/cold – the first time the block is referenced

- Capacity – not enough room in the cache to hold all blocks
  - I.e., this miss would disappear if the cache were big enough

- Conflict – item was replaced because of a location conflict with others
  - I.e., this miss would disappear with higher associativity

# Misses vs. Capacity & Associativity




Increasing associativity reduces miss rate, but has diminishing returns; most benefits are from 1-way to 2/4-way

Miss rates of larger caches are less sensitive to associativity

# Hit/Miss Example

- 2-way associative, 8 blocks, 4 bytes/block, 8-bit address

Access trace

| Address | Hit/Miss? | Type of Miss |
|---|---|---|
| 0101 01 01 | | |
| 0110 10 00 | | |
| 1101 01 10 | | |
| 0101 01 10 | | |
| 1100 01 01 | | |
| 0110 10 00 | | |

Cache content

| | V | Tag | Data | V | Tag | Data |
|---|---|---|---|---|---|---|
| Set 00 | | | | | | |
| Set 01 | | | | | | |
| Set 10 | | | | | | |
| Set 11 | | | | | | |

# Key Design Considerations for Caches

- Where to look up and insert a block? --- Cache organization
- Which block to replace to make room? --- Replacement policy
- How to handle writes? --- Write policy

# Replacement Policy

- On a miss, determine whether to evict an existing entry to make room for the newly requested block, and if so, which entry to evict

- Whether to evict
  - Normal: newly requested block replaces an existing old block
  - Bypass: newly requested block does not enter cache
- Which to evict
  - For direct-mapped, only one choice
  - For set/fully associative, use a replacement policy
  - The existing block chosen to evict is called the victim

# Typical Replacement Policies

- Random: choose one at random; not necessarily a bad policy
- Least Recently Used (LRU): choose the one used least recently
  - Maintain a timestamp for each entry, as the time of the last access
- Least Frequently Used (LFU): choose the one used the least
  - Maintain an aging counter for each entry; increased on hits, halved periodically
- And more …
- Good replacement policies match program locality characteristics

# LRU Example

- Assume a fully-associative cache of 3 blocks that uses LRU replacement,
- An access trace of blocks
  1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- Initial cache is empty

- If accessed data exceed cache capacity, LRU policy could lead to cache thrashing with all misses

| Access | Hit/miss? | Cache content after access |
|---|---|---|
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 1 | | |
| 2 | | |
| 5 | | |
| 1 | | |
| 2 | | |
| 3 | | |
| 4 | | |
| 5 | | |

# How to Handle Writes (Usually)

- A write-hit simply writes data only to cache; defer write to memory until eviction
  - Need a dirty bit in metadata to indicate cached data have been updated
  - When evicting a dirty cache block, write back data to memory

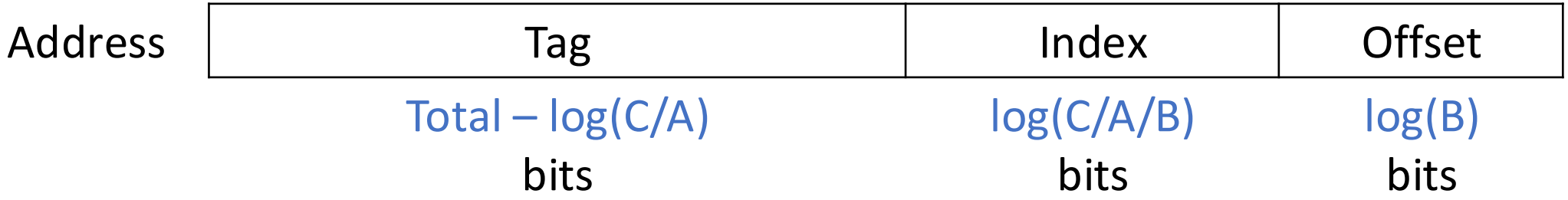
- A write-miss is first treated as a read-miss (fetching block from memory into cache), and then writes to cached block
  - Q: Why we need to first fetch entire block if old data are overwritten?

- The above combination is usually called write-back + write-allocate
- There exist other (less commonly used) choices for writes

# Summary: “ABC” in Cache

- Key configurations: Associativity, Block size, Capacity
  - Both block size and capacity are in bytes

- For the address fields




- For the 3C misses
  - Associativity -> conflict misses
  - Block size -> compulsory misses
  - Capacity -> capacity misses

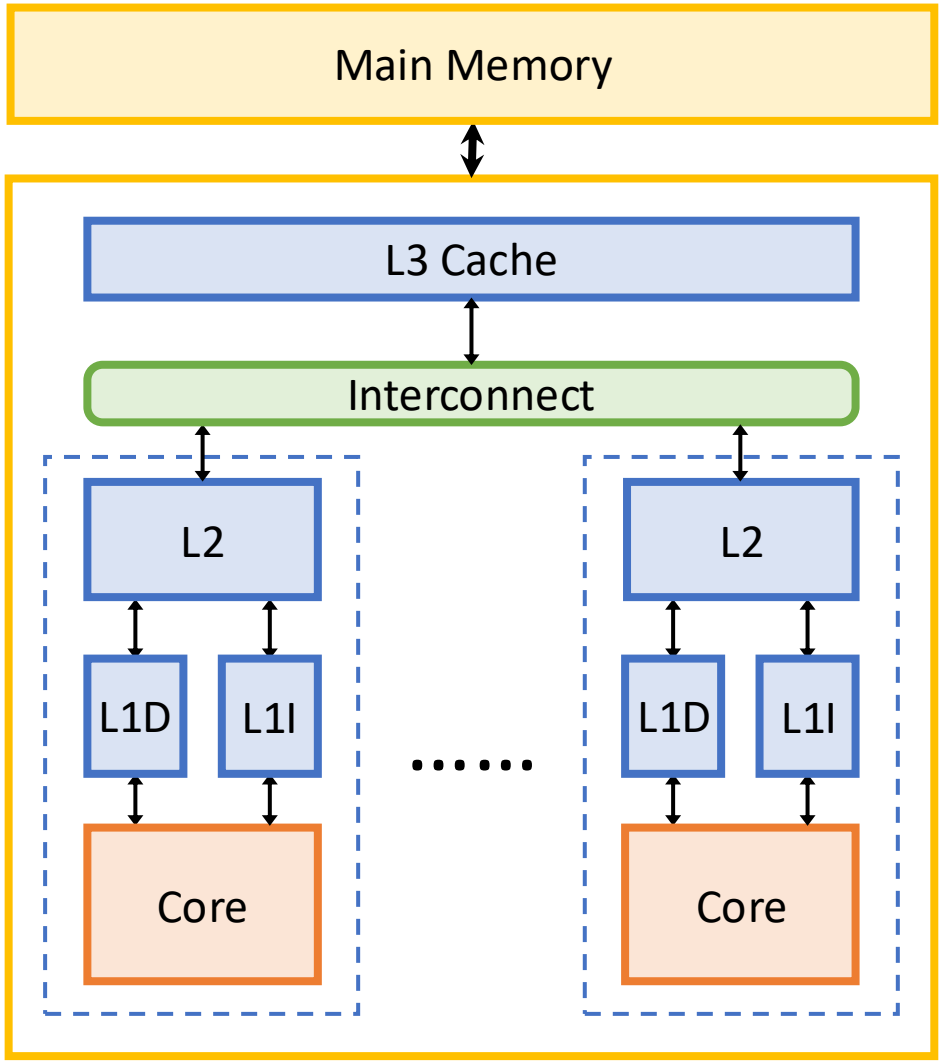
# Today's Outline

- Memory technology choices and performance-capacity tradeoff
- Program locality and the idea of memory hierarchy
- Processor caches
  - Cache performance analysis
  - Cache organization: direct-mapped, fully associative, set associative
  - Replacement policy
  - Write policy
  - ……
- Multi-level cache hierarchy and multi-core caches
- Cache-friendly code optimization

# Multi-Level Caches

- Cache, as a kind of memory, also suffers from the fundamental tradeoff, i.e., between miss rate and hit latency
  - Small caches have low hit latency, while large caches have low miss rate
- To address this tradeoff, multiple levels of caches are often used
  - Level *k* serves as a cache for level *k* + 1
  - A smoothly, gradually changing spectrum of capacity vs. access speed
  - More likely to capture different degrees of locality and dataset sizes
- At each level, AMAT = hit latency + miss rate × miss penalty
  - Miss rate = local miss rate = # misses in this level / # accesses to this level
    = # misses in this level / # misses from the previous level
  - Miss penalty = AMAT of the next level, so AMAT formula is recursively applied

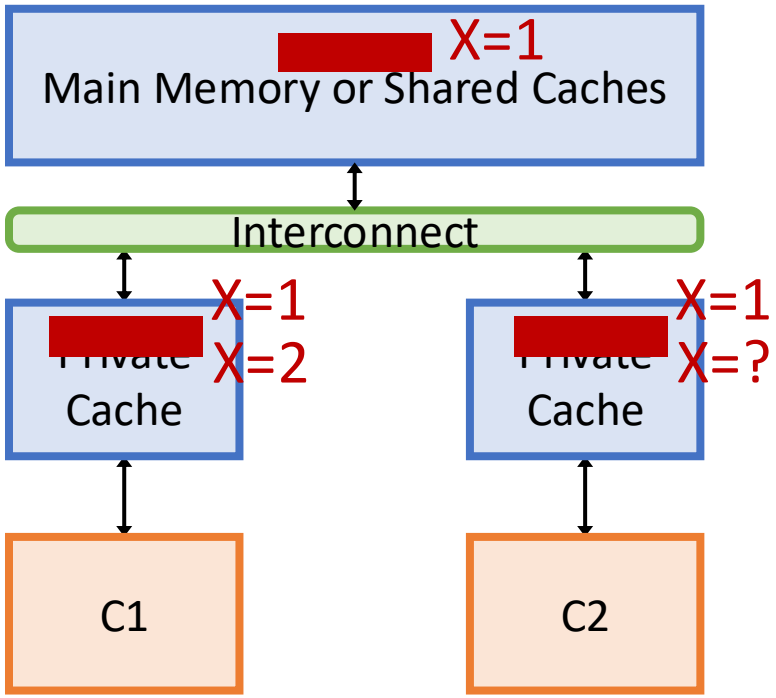
# Typical Multi-Level Caches in Processors




- L1 caches attached to CPU cores
  - Split D-cache & I-cache
  - Small but fast, typically 32 to 64 kB
  - Focus on hit time rather than miss rate
    - E.g., 1 ~ 2 ns access latency
- L2 unified cache per core
  - Typically 256 kB to 1 MB
    - 5 ~ 10 ns access latency
- L3 cache is shared across all cores
  - A.k.a., last-level cache (LLC)
  - Focus on miss rate rather than hit time
    - > 20 ns access latency
- Main memory services misses from L3

# Coherence Problem in Multi-Core Caches

- When each of the multiple cores has a private cache, extra attention is needed to ensure data coherence
- A write by a core to its private cache must also notify other cores' private caches about the data update




C1: load X
C2: load X
C1: store X = X + 1
C2: store X = X + 1

# Coherence Protocols

- Hardware ensures coherence through coherence protocols
  - By sending messages to notify other caches when needed
  - So software does not need to worry about this issue

- Two classes of coherence protocols
  - Update-based: when a write occurs, also propagate the new value to other private caches that has this block
  - Invalidate-based: when a write occurs, invalidate this block (whose value now becomes stale) in other private caches
- Which one is better depends on data sharing characteristics, i.e., whether the other caches may access this block later
- Modern processors ubiquitously use invalidate-based protocols

# Today's Outline

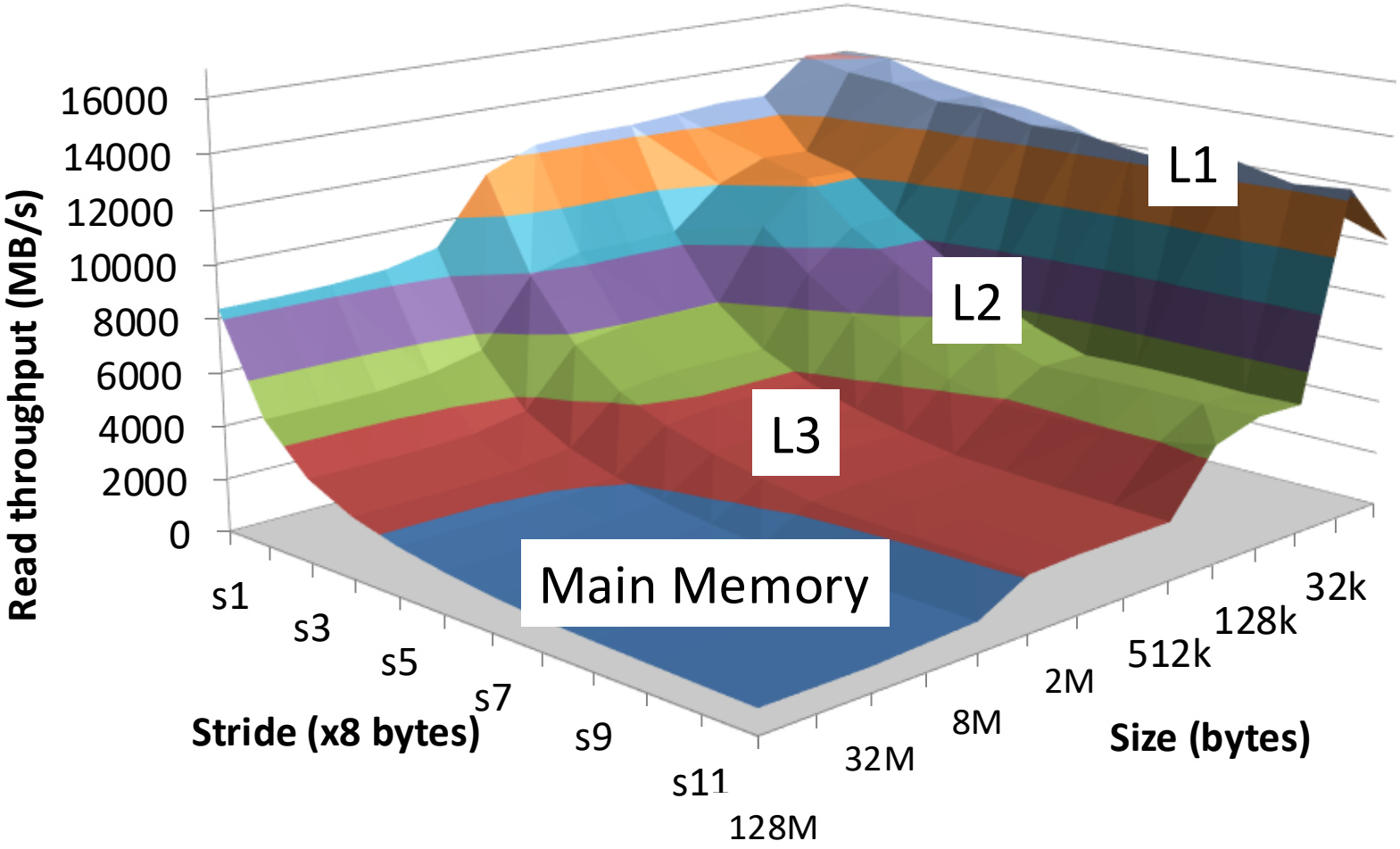
- Memory technology choices and performance-capacity tradeoff
- Program locality and the idea of memory hierarchy
- Processor caches
  - Cache performance analysis
  - Cache organization: direct-mapped, fully associative, set associative
  - Replacement policy
  - Write policy
  - ……
- Multi-level cache hierarchy and multi-core caches
- Cache-friendly code optimization

# Are Caches Transparent to Software?

- In terms of functionality, yes!
  - Processor caches are managed by hardware
    - Software just issues a memory access request using a (main memory) address
    - Hardware finds the closest data copy (in L1, L2, L3, or main memory) to serve the access
  - Cache configurations are not necessarily part of the ISA
  - Same code runs correctly across microarchitectures with different cache designs
- In terms of performance, no!
  - It is important to write cache-friendly code, by exploiting locality
    - E.g., minimize misses in the inner loops (make common case faster)
  - But such performance optimizations are usually not portable across processors
    - Dependent on number of cache levels, cache sizes, replacement policies, etc.
- This is part of Lab 1 that you will need to do

# The Memory Mountain

- Temporal locality: repeated accesses to small amount of data
- Spatial locality: sequential accesses with small strides (e.g., stride-1)




```
while (1) {
  for (i = 0;
       i < SIZE / sizeof(double);
       i++) {
    access A[i * STRIDE];
  }
}
```

# Matrix Multiplication Example

- Multiply two $N \times N$ matrices, $O(N^3)$ total operations
- $N$ reads per source element, $N$ values summed per destination element
- Analyze the number of misses per inner loop
  - Assumptions: 8-byte elements, 64-byte cache block size, $N$ is very large

```
/* ijk */
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    for (k = 0; k < n; k++)
      c[i][j] += a[i][k] * b[k][j];
```

- Key techniques: loop transformations, e.g., reordering, blocking, etc.

# Loop Reordering

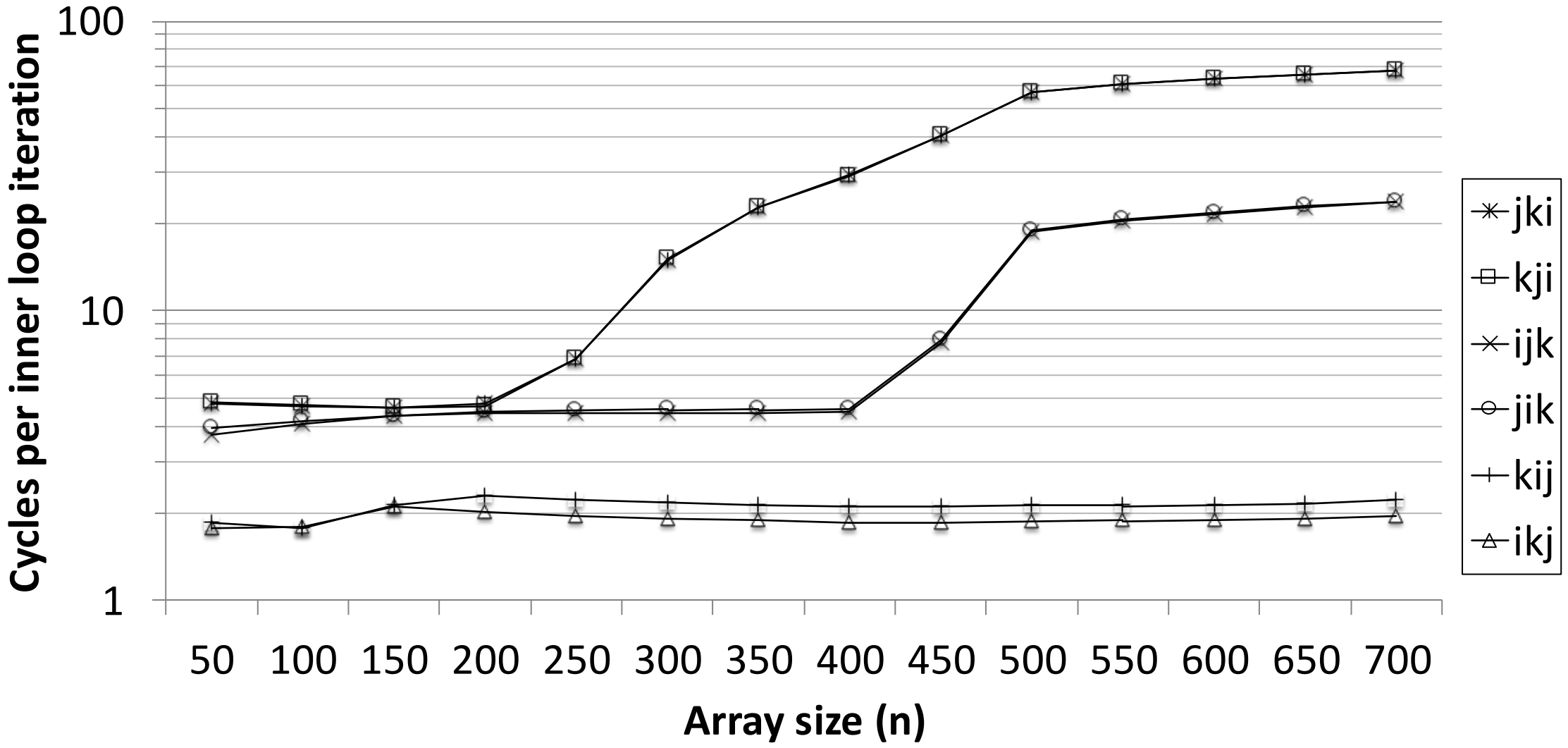
```
/* ijk */
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    for (k = 0; k < n; k++)
      c[i][j] += a[i][k] * b[k][j];
```

```
/* kij */
for (k = 0; k < n; k++)
  for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
      c[i][j] += a[i][k] * b[k][j];
```

```
/* jki */
for (j = 0; j < n; j++)
  for (k = 0; k < n; k++)
    for (i = 0; i < n; i++)
      c[i][j] += a[i][k] * b[k][j];
```

- Pattern ijk and jik
  - a: 1 miss per 8 accesses → 0.125
  - b: 1 miss per access → 1
  - c: ~ all hits → 0
- Pattern kij and ikj
  - a: ~ all hits → 0
  - b: 1 miss per 8 accesses → 0.125
  - c: 1 miss per 8 accesses → 0.125
- Pattern jki and kji
  - a: 1 miss per access → 1
  - b: ~ all hits → 0
  - c: 1 miss per access → 1

# Loop Reordering Performance



# How Can We Do Better?

- Orders kij and ikj are the best so far
  - Exploit temporal locality for a
  - Exploit spatial locality for b and c

- But for all orders, temporal locality only exists for one matrix
  - Elements should be ideally reused *N* times, but evicted due to limited capacity
- How can we make the large matrices fit in the small cache?

# Loop Blocking

- Loop blocking (a.k.a., loop tiling)
  - Split matrices into smaller tiles ($B \times B$) that fit in cache
  - Reuse data in the tile as much as possible before going to the next tile
  - Improve temporal locality for all matrices

```
for (kk = 0; kk < n; kk += B)
 for (ii = 0; ii < n; ii += B)
  for (jj = 0; jj < n; jj += B)
   for (k = kk; k < kk + B; k++)
    for (i = ii; i < ii + B; i++)
     for (j = jj; j < jj + B; j++)
      c[i][j] += a[i][k] * b[k][j];
```

- Multiple levels of loop blocking can be applied for multi-level caches

# Loop Blocking Performance

- Without blocking (assume order kij)
  - $N^2$ elements in each of a, b, c; each element is accessed $N$ times
  - Misses per access for a, b, c elements = 0, 1/8, 1/8
  - Total misses = $(1/8 \times N^2 + 1/8 \times N^2) \times N = N^3/4$

- With blocking (also assume order kij)
  - $(N/B)^2$ blocks in each of a, b, c; each block is accessed $N/B$ times
  - Assume three $B \times B$ tiles (for a, b, c) all fit in cache
    - ~ all hits for a tiles (reused across multiple b, c tiles)
    - $B^2/8$ misses for each b, c tile (first word in each cache block)
  - Total misses = $(B^2/8 + B^2/8) \times (N/B)^2 \times N/B = N^3/4B$

- We should use the largest $B$ where $3B^2 < C$

# Loop Transformations in Real World

- Widely used in high-perf math libraries, e.g., BLAS, cuBLAS, Intel MKL, …
  - Q: What are the shortcomings of loop transformations?

- Ideally, the compiler should do such optimizations automatically for us
- Halide [PLDI'13], halide-lang.org
  - A domain-specific language that describes loop transformations at high level
  - Define a set of schedule primitives for loops, e.g., `split`, `reorder`, `unroll`, …
  - Originally designed for image processing, but later also handle ML/NN
- TVM [OSDI'18], AutoTVM [NeurIPS'18], tvm.apache.org
  - A compiler framework for tensor computations in ML/NN
  - Use ML/NN algorithms to search the design space
  - Under the hood it uses a similar language as Halide to describe the design space

# Summary

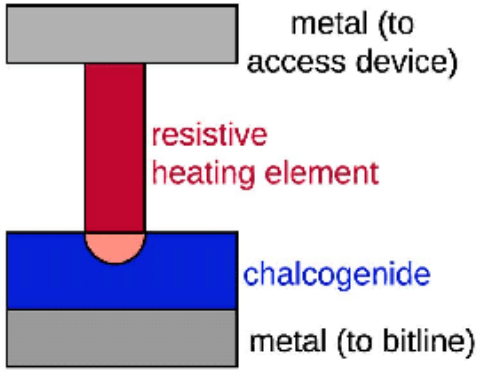
- Memory technologies: SRAM and DRAM
- Locality: temporal and spatial
- Memory hierarchy to exploit locality in programs
  - Combine fast, small memories with slow, large memories, to be fast & large
- AMAT = hit latency + miss rate × miss penalty
- Associativity: direct mapped, fully associative, set associative
- 3Cs of misses: compulsory, capacity, conflict
- Replacement policies and write policies
- Multi-level caches
- Multi-core caches and cache coherence
- Cache-friendly code optimization with loop transformations

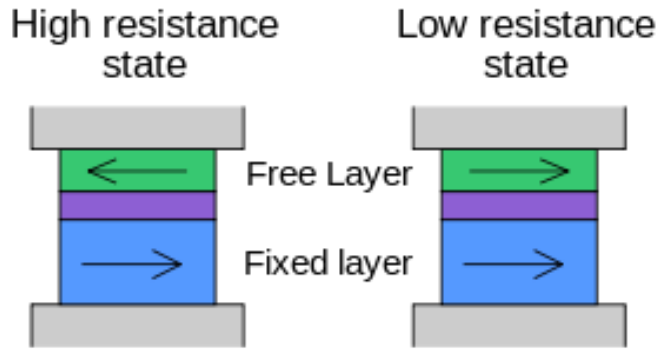
# Next Lecture: Processes/Threads, and How to Schedule Them

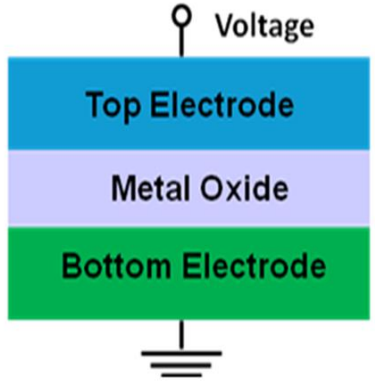
# Appendix

# Side Topic: New Non-Volatile Memories

- Phase change memory (PCM), spin-transfer torque RAM (STT-RAM), resistive RAM (ReRAM, a.k.a., memristor), …
  - Physical state changes (e.g., amorphous vs. crystalline, magnetic orientation of spin, metal-insulator-metal crosspoints) cause resistance changes




- Characteristics
  - ✓ Non-volatile; could also be used as secondary storage
  - ✓ More dense than DRAM → even larger capacity
  - ✗ Slow to access due to higher energy to change physical states